

CompoundIQ — System Architecture Overview

Version: 2.0 | **Date:** April 9, 2026 **Status:** Production POC — 19 phases, 86 work orders (81 completed, 5 backlog)

1. Product Overview

CompoundIQ is a B2B2C functional medicine infrastructure platform that connects independent prescribing clinics with compounding pharmacies through intelligent sourcing, automated fulfillment, and transparent pricing.

The Problem: Independent clinics prescribe compounded medications at scale but lack infrastructure to do so profitably. Sourcing is manual (phone calls, spreadsheets), margin math is error-prone, and fulfillment is entirely fax-dependent with no tracking or escalation.

The Solution: A platform that lets a medical assistant search all licensed pharmacies in seconds, lock a margin, and generate a tokenized SMS checkout link. Upon payment, the system auto-generates a compliant prescription and routes it to the pharmacy via the optimal integration channel.

Revenue Model: Per-transaction platform fee captured from the spread between wholesale and retail price. The clinic earns margin on every fill. The pharmacy gets structured, error-free orders. The patient gets seamless guest checkout via SMS.

2. Three Applications, One Codebase

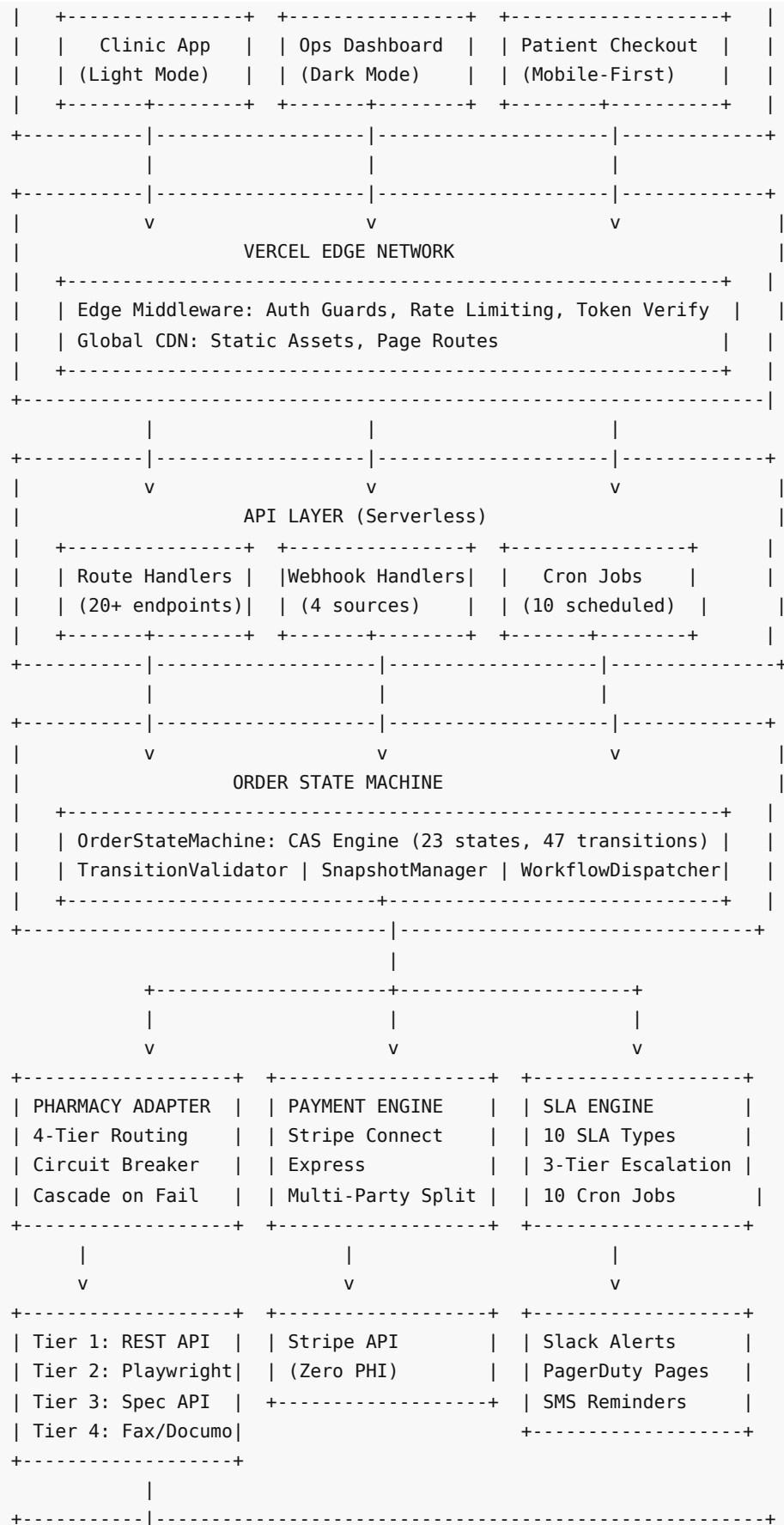
CompoundIQ consists of three distinct web applications within a single Next.js 16 codebase, organized via App Router route groups:

Application	Route Group	Users	Theme	Viewport
Clinic App	/ (clinic-app)/	Medical Assistants, Providers, Clinic Admins	Light mode	Desktop
Ops Dashboard	/ (ops-dashboard)/	Internal operations team	Dark mode	Desktop
Patient Checkout	/checkout/	Patients (guest, no login)	Light mode	Mobile-first (320–428px)

Each application has its own layout, auth guard, error boundaries, and loading states. There is no cross-navigation between applications. Role-based access control is enforced at both the middleware and database level.

3. System Architecture Diagram





v DATA LAYER		
+-----+		
Supabase (PostgreSQL 15+)		
33 Tables + 1 View 10 Enums Row-Level Security on ALL tables		
+-----+		
Supabase Vault	Supabase Auth	Supabase Storage
AES-256-GCM	JWT Custom Claims	RLS-Gated Buckets
Pharmacy Creds	Role-Based Access	Fax PDFs, Screenshots
+-----+		
+-----+		

4. Technology Stack

Layer	Technology	Version / Notes
Frontend	Next.js (App Router)	16+ with React 19, Server + Client Components
UI Framework	Tailwind CSS + Shadcn/UI	Semantic design tokens, dark mode support
Backend / Database	Supabase (PostgreSQL)	15+ with RLS on all 33 tables, Supabase Auth
Authentication	Supabase Auth + JWT	Custom claims: app_role, clinic_id in user_metadata
Payments	Stripe Connect Express	Multi-party split: patient pays, clinic receives payout, platform captures fee
Fax	Documo mFax REST API	HIPAA-compliant, Tier 4 fallback
SMS	Twilio Programmable Messaging	Delivery tracking, opt-out handling
Portal Automation	Playwright	Headless browser for Tier 2 pharmacy portals
Credential Storage	Supabase Vault	AES-256-GCM encryption for all pharmacy credentials
Cron Scheduling	Vercel Crons	10 scheduled jobs defined in vercel.json
Deployment	Vercel	Serverless functions, atomic deploys, instant rollback
Monitoring	Sentry + Slack + PagerDuty	Error tracking (PHI-scrubbed), ops alerts, critical escalation

5. The 4-Tier Pharmacy Adapter Layer

The Pharmacy Adapter Layer is the platform's core technical differentiator — a unified integration engine that abstracts pharmacy connectivity behind a single interface, enabling seamless order routing regardless of each pharmacy's technical capability.

Tier	Method	Pharmacies	Speed	Coverage
Tier 1	Direct REST API	ReviveRX, Vios, MediVera, LifeFile network (Empower, Belmar, UCP, Strive), Precision	Instant (~2s)	~25%
Tier 2	Portal Automation (Playwright)	Olympia/DrScript, Wells/WellsPx3	~5 min	~20%
Tier 3	Standardized API Spec	Future partners adopting our published spec	Instant	Future 30%+
Tier 4	Fax Fallback (Documo)	Any pharmacy without Tier 1/2/3 capability	~30 min	Universal

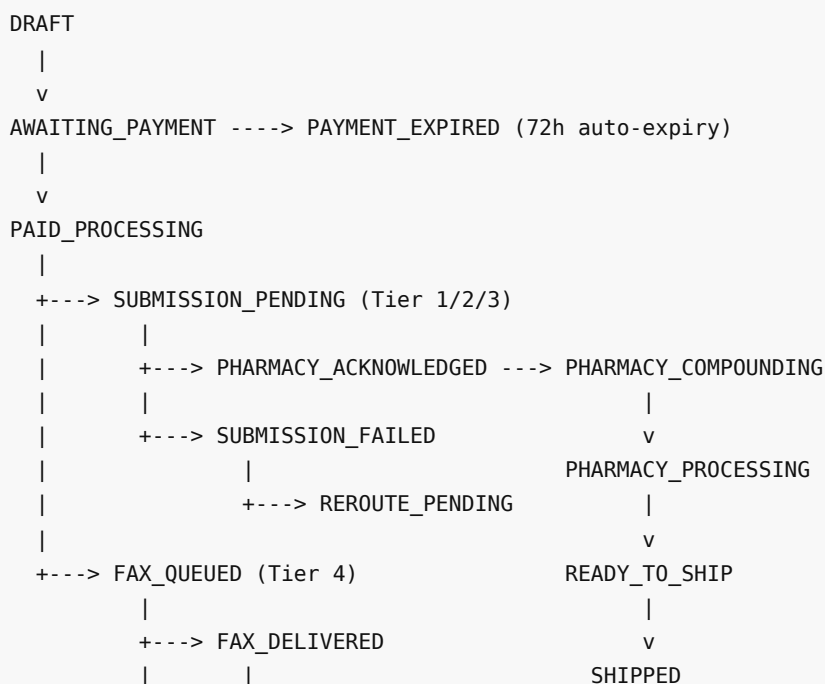
Key architectural insight: LifeFile is the de facto B2B standard for compounding pharmacies. A single LifeFile API integration unlocks Empower, Belmar, UCP, and Strive — the largest pharmacy network in the country.

Routing Logic

- **Deterministic:** Always selects the highest available tier for each pharmacy
- **Cascade on failure:** Tier 1 fails -> Tier 2 (if available) -> Tier 4 (fax)
- **Circuit breaker:** 3 consecutive failures opens the breaker, 5-minute cooldown, 2 consecutive successes to close (HC-15)
- **Audit trail:** Every submission attempt logged to adapter_submissions table

6. Order Lifecycle — 23-State Machine

Every order flows through a closed state machine with exactly 23 states and 47 valid transitions. All transitions use atomic Compare-And-Swap (CAS) to prevent race conditions.



```

+---> FAX_FAILED
      |
      +---> REROUTE_PENDING
      |
      v
      DELIVERED (terminal)

```

Terminal states: CANCELLED, REFUNDED, DELIVERED **Error states:** ERROR_PAYMENT_FAILED, ERROR_COMPLIANCE_HOLD **Financial recovery:** DISPUTED, REFUND_PENDING

Snapshot Immutability

When an order transitions from DRAFT to AWAITING_PAYMENT, 7 fields are permanently frozen:

- wholesale_price_snapshot, retail_price_snapshot
- medication_snapshot, shipping_state_snapshot
- provider_npi_snapshot, pharmacy_snapshot
- provider_signature_hash_snapshot

A PostgreSQL trigger (`prevent_snapshot_mutation()`) blocks any subsequent UPDATE to these fields. There is no unlock mechanism — to change a locked field, the order must be cancelled and a new one created.

7. Payment Architecture — Stripe Connect Express

Patient pays \$300 (retail_price)

```

      |
      v
+-----+
| Stripe Connect |
| Express        |
+-----+
      |
      +----+----+
      |          |
      v          v
Platform    Clinic
Fee: $22.50 Payout: $127.50
(15% of    (via transfer_data
margin)    .destination)

```

- **Single PaymentIntent** per order — never multiple
- **Automatic 3-way split:** Patient pays retail, platform captures fee via `application_fee_amount`, clinic receives payout via `transfer_data.destination`
- **Zero PHI in Stripe:** Metadata contains `order_id` only. Description: "Custom compounded prescription order." No medication names, patient names, or NPI numbers.
- **Stripe processing fees** (2.9% + \$0.30) deducted from the platform's portion
- **Transfer failures** do NOT affect order status — they are financial reconciliation events

8. SLA Engine — 10 Enforcement Types

The SLA engine is the platform's time-enforcement backbone. A Vercel Cron job runs every 5 minutes scanning for breached deadlines and triggering automatic escalation.

SLA Type	Deadline	Trigger
PAYMENT_EXPIRY	72 hours	Order enters AWAITING_PAYMENT
SMS_REMINDER_24H	24 hours	Checkout link sent
SMS_REMINDER_48H	48 hours	Checkout link sent (24h before expiry)
FAX_DELIVERY	30 minutes	Fax queued via Documo
PHARMACY_ACK	4 business hours	Fax delivered (Tier 4)
PHARMACY_COMPOUNDING	TAT + 4h buffer	Pharmacy acknowledges
SHIPPING	24 hours	Ready to ship
TRACKING_UPDATE	24 hours	Order shipped
ADAPTER_SUBMISSION_ACK	5-30 min (tier-dependent)	Adapter submission triggered
PHARMACY_COMPOUNDING_ACK	2 business hours	Pharmacy acknowledges (Tier 1/2/3)

3-Tier Escalation

Tier	Channel	Trigger
Tier 0	Automated action (retry, cascade)	At breach
Tier 1	Slack #ops-alerts	At breach (or after Tier 0)
Tier 2	Slack DM to ops lead	15 min unacknowledged
Tier 3	PagerDuty page	30 min unacknowledged

9. Webhook Architecture — 4 External Sources

All webhooks follow a 7-step processing pipeline: Receive -> Authenticate -> Extract event_id -> Idempotency check -> Process -> Record outcome -> Respond 200.

Source	Events	Auth Method
Stripe	payment_intent.succeeded, charge.dispute.created, transfer.failed, account.updated	HMAC-SHA256 (whsec_*), 5-min timestamp tolerance
Documo	fax.delivered, fax.failed, fax.received (inbound)	Token-based verification
Twilio	message.status (delivered, failed, undelivered)	X-Twilio-Signature HMAC-SHA1
Pharmacy APIs	order.acknowledged, order.compounding, order.rejected, order.shipped	Per-pharmacy HMAC-SHA256 via Supabase

		Vault
--	--	-------

Critical invariant: The system ALWAYS returns HTTP 200 to webhook senders — even on internal failures. This prevents retry storms. Failures are handled via the internal Dead Letter Queue.

10. Database Schema

33 Tables with Full Row-Level Security

Core Entities (12 tables): clinics, providers, patients, pharmacies, pharmacy_state_licenses, catalog, catalog_history, orders, order_status_history, webhook_events, order_sla_deadlines, inbound_fax_queue

Pharmacy Adapter Layer (5 tables): pharmacy_api_configs, pharmacy_portal_configs, adapter_submissions, normalized_catalog, pharmacy_webhook_events

Hierarchical Medication Catalog (8 tables — WO-82): ingredients, salt_forms, dosage_forms, routes_of_administration, formulations, formulation_ingredients, pharmacy_formulations, sig_templates

Provider Speed Features (3 tables — WO-85): provider_favorites, protocol_templates, protocol_items

Regulatory Compliance (5 tables — WO-86): epcs_audit_log, drug_interactions, patient_protocol_phases, phase_advancement_history + TOTP columns on providers (totp_secret_encrypted, totp_enabled, totp_verified_at)

Views: provider_prescribing_history (aggregates order data for adaptive shortlist)

10 Enum Types

order_status_enum (23 values), stripe_connect_status_enum (4), cancellation_reason_enum (5), regulatory_status_enum (3), webhook_source_enum (4), sla_type_enum (10), inbound_fax_status_enum (5), integration_tier_enum (4), adapter_submission_status_enum (8), catalog_source_enum (3)

Key Schema Principles

- All monetary values: NUMERIC(10,2) — never floating point
 - Soft-delete on all tables (deleted_at + is_active) — no physical DELETES
 - Append-only audit tables for all state transitions (order_status_history, epcs_audit_log, phase_advancement_history)
 - TOTP secrets encrypted with AES-256-GCM at rest
 - Supabase Vault for all pharmacy credentials (UUID references, never plaintext)
 - Foreign keys enforced everywhere
 - Hierarchical catalog: ingredients → salt_forms → formulations → pharmacy_formulations
-

11. Infrastructure — 10 Vercel Cron Jobs

Endpoint	Schedule	Purpose
/api/cron/sla-check	Every 5 min	SLA breach scanning and escalation

/api/cron/sla-refire	Every 5 min	Re-escalate stalled SLA deadlines
/api/cron/payment-expiry	Every 15 min	Auto-expire unpaid orders at 72 hours
/api/cron/adapter-health-check	Every 10 min	Circuit breaker state updates
/api/cron/submission-reconciliation	Every 30 min	Detect stuck submissions
/api/cron/portal-status-poll	Every 30 min	Tier 2 portal status polling
/api/cron/fax-retry	Every 5 min	Fax retry with exponential backoff
/api/cron/screenshot-cleanup	Every hour	Remove expired Tier 2 screenshots
/api/cron/daily-digest	14:00 UTC daily	Ops pipeline health summary to Slack
/api/cron/poc-credential-sync	05:00 UTC daily	Reset POC demo accounts to canonical passwords

All cron endpoints are protected by CRON_SECRET bearer token authentication.

12. Security & HIPAA Compliance

Data Protection

Control	Implementation
Data at rest	AES-256 encryption (Supabase-managed)
Data in transit	TLS 1.2+ on all connections
Credential storage	Supabase Vault (AES-256-GCM) — never plaintext
Access control	Row-Level Security on all 33 tables
EPCS 2FA	TOTP authenticator (DEA 21 CFR 1311) for controlled substances
TOTP secrets	AES-256-GCM encrypted at rest (not plaintext)
Session management	30-min idle timeout, 12-hour absolute max
Audit trail	Append-only tables for all state transitions + EPCS audit log
Drug interactions	Known interaction pairs checked at review with severity alerts
PHI in Stripe	ZERO — metadata contains order_id only
PHI in SMS	Minimal — first name + URL only, never medication names
PHI in monitoring	Sentry beforeSend scrubs all PHI before transmission
Supabase Realtime	DISABLED — hard HIPAA requirement (leaks PHI via WebSocket)

Authentication Model

Context	Method	Access Level
Clinic users	Supabase Auth JWT (app_role + clinic_id)	Own clinic data only
Ops admins	Supabase Auth JWT (app_role = ops_admin)	Cross-clinic read access
Patients	Stateless JWT in URL (72h expiry, HS256)	Single order only
Webhooks	Service-specific signature verification	Service role (bypasses RLS)
Cron jobs	CRON_SECRET bearer token	Service role

Role-Based Access Control (4 roles)

Role	Clinic App	Ops Dashboard	Patient Checkout
clinic_admin	Full access (own clinic)	Blocked	N/A
provider	Signature + review only	Blocked	N/A
medical_assistant	Full workflow (own clinic)	Blocked	N/A
ops_admin	Blocked	Full access (all clinics)	N/A
patient (anon)	Blocked	Blocked	Single order via JWT

13. Key Metrics

Metric	Value
PostgreSQL tables	33
Enum types	10
Order statuses	23 (with 47 valid transitions)
Hard constraints	16 (HC-01 through HC-16)
SLA types	10
Integration tiers	4
Cron jobs	10
Build phases completed	19
Work orders completed	87 (all merged to main)
Applications	3
User roles	4 (+ patient anonymous)
Webhook sources	4 (Stripe, Documo, Twilio, Pharmacy APIs)

Drug interaction pairs	6 (seeded, expandable)
Provider favorites	Per-provider saved prescription configs
Protocol templates	Per-clinic multi-medication bundles
Environment variables	40+ across 8 categories
Automated tests	65 Playwright E2E (Chromium / Firefox / WebKit / mobile Chrome) + 6 jest unit tests. E2E isolated on dedicated Supabase project. CI gates every merge.

14. Deployment Architecture

```

GitHub (master branch)
  |
  v
Vercel CI/CD
(Automatic on push)
  |
+----+----+
|      |      |
v      v
Production Preview
(atomic (per PR,
deploy) staging
       creds)

```

- **Git-driven:** Every push to master auto-deploys to production
- **Atomic deploys:** Old version serves traffic until new version is fully ready
- **Instant rollback:** One-click restore to any prior deployment
- **Preview environments:** Every PR gets an isolated preview URL with staging credentials
- **Zero-downtime migrations:** 5-phase pattern (additive -> dual-write -> backfill -> tighten -> cleanup)

*Built with Next.js 16, Supabase, Stripe Connect Express, Twilio, Documo mFax, and Playwright.
 Deployed on Vercel serverless infrastructure. Designed for HIPAA-adjacent compliance with zero PHI exposure to non-covered entities.*